

## Homework 3:

### User Models and JavaScript

Warm Up Due: Friday Feb 06, 2026 11:59 PM on Courseworks  
Main Due: Monday Feb 09, 2026 11:59 PM on Courseworks  
HW3 will be accepted as on time until Tuesday Feb 10, 2026 8:00 AM

### Week 03 Warm-Up:

#### What to submit:

2 separate files with the following titles:

- my\_ta\_section.pdf
- js\_warmup\_screenshot.pdf
- js\_warmup.html

Submit them separately to Courseworks, **do not zip them up**.

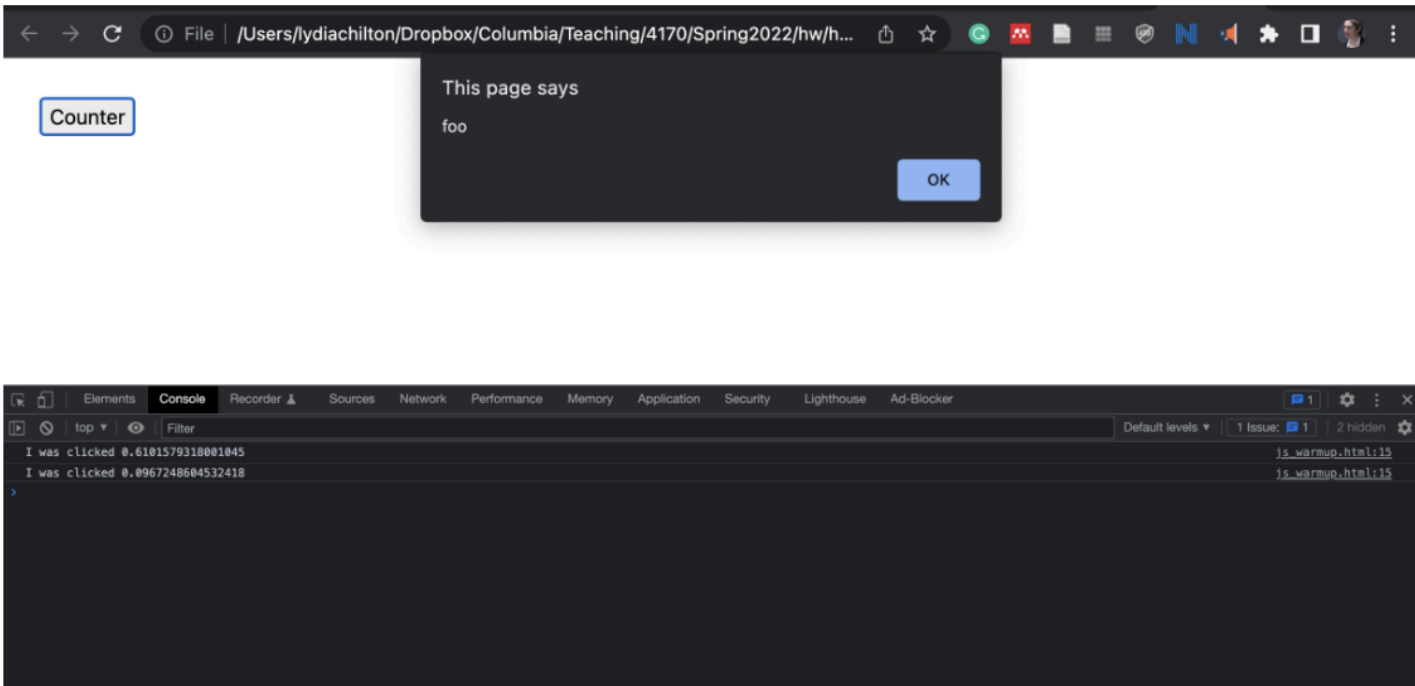
#### Problems:

1. Sign up for a TA section (See Courseworks announcement with link to a Google Sheet)
  - a. Which TA did you select?
  - b. Who else (classmates) is in your group?
2. Create a webpage with a single button that has the text "counter" on the button. When the user presses it, two things happen (in this order):
  - a. The JavaScript console has a message in it that says *"I was clicked"* and has a random number.
  - b. An alert box appears with the following text, "I am alert!".

You must use jQuery (not plain JavaScript or HTML) to register the click handler after the page has loaded. Use a single HTML file that has all your HTML, and JavaScript code (and CSS if you use any). Use the following snippet to include jQuery:

```
<script src=https://code.jquery.com/jquery-3.3.1.min.js  
crossorigin="anonymous"></script>
```

3. Submit a screenshot showing the alert and at least two console statements. It should look like the following:



## Week 03 Main:

### What to submit:

Two files and a link to a youtube video demoing the functionality and explaining how it works.

- user\_model\_questions.pdf
- tweet\_post.html

### Problem 1. User Models

1. One of the primary goals of Courseworks is to allow students to upload assignments. Starting from <https://courseworks2.columbia.edu/> what are the subgoals needed to complete that goal. For each stage, take a screenshot, give the stage a name that describes that subgoal, and name one or two low-level actions that are critical to completing that subgoal.
  - a. When the final subgoal is complete, what feedback is given to let you know you executed the goal correctly?
  - b. Note: there is no exact number of subgoals. As a general heuristic, usually each page has one subgoal. But sometimes a single page has two subgoals. Use your best judgement. Talk to a TA if you're not sure.
2. Many mobile devices (phones, tablets) have menus that appear when you swipe from the top, rather than having to click a start button or a menu button. What advantage does it have from the perspective of low-level interactions? Why?
3. What major features did every major streaming service (YouTube, Netflix, Hulu, etc.)

introduce to save users one low-level interaction? Why did they introduce it? (You may answer from the perspective of users or the service owners, or both? What do you think? Do you use this feature or did you disable it? Why or why not?

4. Name of application not mentioned in class or on this assignment that you use that has eliminated or simplified its interactions to reduce friction for users. Name the application and write 2-3 sentences explaining how the interaction was simplified and why you like it.

## Problem 2. Recreate functionality to post a tweet.

Create a website with HTML, CSS, JavaScript and jQuery where a user can write a tweet and see it appear below in a list of their own tweets. This is similar to the functionality of Twitter described in class regarding execution and evaluation. Don't replicate the Twitter design, just the basic functionality of writing and posting a tweet with a **max of 30 characters** and providing feedback to help users evaluate the goal of posting a tweet and the subgoal of writing a tweet that is within the max number of characters.

**The following is the functionality you need to replicate:**

1. There needs to be a place to write the tweet and a button to press to post the tweet.
2. There needs to be a number that shows how many characters the user has remaining (call it **.characters\_remaining**). The max length can be hard coded in JavaScript (but not in HTML).
3. As the user types their tweet text, the following must happen:
  - a. The **.characters\_remaining** number must change to reflect the current number of characters left.
  - b. If the user exceeds the number of characters, a negative number must be shown for **.characters\_remaining**.
  - c. Negative **.characters\_remaining** numbers must be red and bold
  - d. Non-negative **.characters\_remaining** numbers must be lightgrey and normal font weight.
  - e. If the number of **.characters\_remaining** is negative, the "post tweet" button must not be clickable (disabled).
  - f. If the user presses "enter" the "post tweet" action is taken.
4. When the user presses the "post tweet" button the following things must happen:
  - a. The post must appear with the poster's username. The username can be hard coded in JavaScript (but not in HTML).
  - b. New posts must appear at the top of the list of posts, so that users see the latest tweets at the top of the list.
  - c. The text in the text input element must disappear.
  - d. The number that counts the number of characters remaining must return to the maximum number.

- e. The cursor must return to the box where the user writes tweets (so that they can immediately start writing another tweet!)
- f. If there is no text in the text input element, or all the text is blank space, do not post the tweet.

Obviously, “posting a tweet” will only work locally on your computer. It will not post anything to the internet. Also, the posted tweets will not persist if you close the webpage and open it again.

For this problem you need to turn in your code, and a short ~2min video of yourself explaining the application while you use it, showing relevant code or things in the Developer Tools, as necessary.

You do not need to use Bootstrap, but if you want to use it, please use the same link from previous assignments.

This assignment is about functionality and not design. As long as we can see and read all the elements and they have appropriate sizes and positions, the design is fine. Below is a screenshot of a reference solution. You do not need to copy it, it is only here to convey that having a very basic design is fine. The following screenshots show examples of what the app looks like in various stages. You may copy this basic design if you like. Again, it’s not about design, it’s about functionality.

**The following is a mockup of the design you need to replicate:**

**State: The user is able to post a tweet**

Write a tweet

Call me Ishmael.

14 Post Tweet

POSTS

|         |              |
|---------|--------------|
| chilton | Third post.  |
| chilton | Second post. |
| chilton | First post.  |

**State: The user is unable to post a tweet** (the text continues past where you can read it)

Write a tweet

Call me Ishmael. Some years ago- never

**-12** Post Tweet

POSTS

|                |             |
|----------------|-------------|
| <b>chilton</b> | Third post  |
| <b>chilton</b> | Second post |
| <b>chilton</b> | First post  |